

Unit 5 - Transaction and Concurrency Control (CO5)

Topics:

Transactions and ACID Properties, Transaction States and Types, Schedules and Serializability, Concurrency Control Protocols, Deadlock Management, Recovery and Logging Techniques, Commands: Commit, Rollback, SavePoint

Topics to be covered

- ▶ Introducing Transactions,
- ▶ Processing a transaction and State of transaction,
- ▶ ACID property of a transaction

Single-User Versus Multiuser system

- ▶ Single-User:
 - ▶ At most one user at a time can use the system
 - ▶ Mostly restricted to personal computer system
- ▶ Multi-User system:
 - ▶ Many users can use the system, and access the database concurrently
 - ▶ Eg. Airline reservation system, banking system, stock market, supper market etc...
 - ▶ In this system hundred and thousands of users are operating on the database by submitting transactions concurrently to the system.

Multiprogramming and Parallel Processing

- ▶ Multiprogramming allows the operating system of the computer to execute multiple programs / processes at the same time
- ▶ A single CPU can execute only one process at a time
- ▶ Multiprogramming operating system executes some command from one process, then suspend the process and execute commands from next process...and so on
- ▶ A process is resumed at the point where it was suspended, whenever it gets turn to use CPU again
- ▶ If the computer has multiple CPUs, parallel processing of multiple processes is possible
- ▶ Most of the systems have interleaved concurrency - i.e. primary resource accessed concurrently by interactive users or application programs, which are constantly retrieving information from / and modifying the DB.

Multiprogramming and Parallel Processing

Difference Between Multiprogramming and Parallel Processing

Feature	Multiprogramming	Parallel Processing
Concept	Multiple programs kept in memory, CPU switches between them	Multiple CPUs/cores execute processes simultaneously
Execution	Only one runs at a time (others wait)	Many run at the same time
Goal	Keep CPU busy, reduce idle time	Speed up execution, increase throughput
Hardware need	Single CPU sufficient	Requires multiple CPUs or multi-core CPU
Example	Running MS Word + Music Player on a single-core CPU	Running video rendering split across 8 cores

Multiprogramming and Parallel

Multiprogramming vs Parallel Processing Examples

Category	Operating System Examples	How it works (Example)
Multiprogramming OS	- UNIX (classic) - MS-DOS with memory managers - Windows (XP, 7, 10 on single-core CPU) - Linux (on single-core CPU)	- CPU switches between MS Word, Browser, and Music Player → only one runs at a time, but all are kept in memory. - Example: On a single-core Linux system, CPU executes process A, then B, then C in rapid sequence.
Parallel Processing OS	- Linux with SMP (Symmetric Multiprocessing) - Windows Server editions - macOS - Android / iOS (on multi-core processors)	- Multiple processors/cores execute different tasks simultaneously. - Example: On a quad-core machine: • Core 1 → Chrome • Core 2 → Spotify • Core 3 → Word • Core 4 → Antivirus scan - OR a single app like video rendering splits its job across all cores.

Key Takeaway:

- **Multiprogramming** → Single CPU, illusion of parallelism (time-sharing).
- **Parallel Processing** → Multi-core/multi-CPU, *true* parallel execution.

Transactions, Database Items, Read and Write Operations and DBMS Buffers

- ▶ A transaction is an executing program that forms a logical unit of database processing.
- ▶ A transaction includes one or more database access operations—these can include insertion, deletion, modification (update), or retrieval operations.
- ▶ One way of specifying the transaction boundaries is by specifying explicit begin transaction and end transaction statements in an application program
- ▶ A single application program may contain more than one transaction if it contains several transaction boundaries
- ▶ If the database operations in a transaction do not update the database but only retrieve data, the transaction is called a read-only transaction; otherwise it is known as a read-write transaction.
- ▶ A database is basically represented as a collection of named data items. The size of a data item is called its granularity.
- ▶ A data item can be a database record, but it can also be a larger unit such as a whole disk block, or even a smaller unit such as an individual field (attribute) value of some record in the database

Access operations that a transaction can include are as follows:

- ▶ `read_item(X)`
 - ▶ Reads a database item named X into a program variable. To simplify our notation, we assume that the program variable is also named X.
- ▶ `write_item(X).`
 - ▶ Writes the value of program variable X into the database item named X

- ▶ Executing a `read_item(X)` command includes the following steps:
 - ▶ 1. Find the address of the disk block that contains item X.
 - ▶ 2. Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer). The size of the buffer is the same as the disk block size.
 - ▶ 3. Copy item X from the buffer to the program variable named X.
- ▶ Executing a `write_item(X)` command includes the following steps:
 - ▶ 1. Find the address of the disk block that contains item X.
 - ▶ 2. Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
 - ▶ 3. Copy item X from the program variable named X into its correct location in the buffer.
 - ▶ 4. Store the updated disk block from the buffer back to disk (either immediately or at some later point in time).

Transaction and System Concept

- ▶ A transaction is an atomic unit of work that should either be completed in its entirety or not done at all.
- ▶ For recovery purposes, the system needs to keep track of when each transaction starts, terminates, and commits, or aborts
- ▶ The recovery manager of the DBMS needs to keep track of the following operations:
- ▶ **BEGIN_TRANSACTION**. This marks the beginning of transaction execution.
- ▶ **READ or WRITE**. These specify read or write operations on the database items that are executed as part of a transaction.
- ▶ **END_TRANSACTION**. This specifies that READ and WRITE transaction operations have ended and marks the end of transaction execution. However, at this point it may be necessary to check whether the changes introduced by the transaction can be permanently applied to the database (committed) or whether the transaction has to be aborted because it violates serializability or for some other reason.
- ▶ **COMMIT_TRANSACTION**. This signals a successful end of the transaction so that any changes (updates) executed by the transaction can be safely committed to the database and will not be undone.
- ▶ **ROLLBACK (or ABORT)**. This signals that the transaction has ended unsuccessfully, so that any changes or effects that the transaction may have applied to the database must be undone.

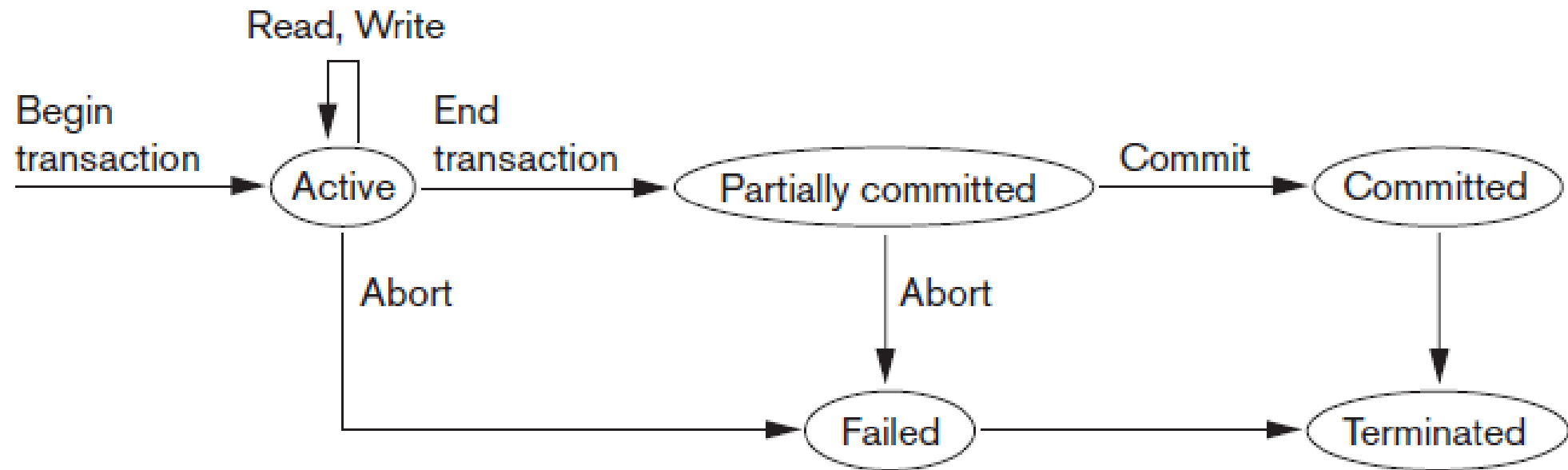


Figure 20.4

State transition diagram illustrating the states for transaction execution.

Desirable Properties of Transactions

- ▶ **The following are the ACID properties:**
- ▶ **Atomicity.** A transaction is an atomic unit of processing; it should either be performed in its entirety or not performed at all.
- ▶ **Consistency preservation.** A transaction should be consistency preserving, meaning that if it is completely executed from beginning to end without interference from other transactions, it should take the database from one consistent state to another.
- ▶ **Isolation.** A transaction should appear as though it is being executed in isolation from other transactions, even though many transactions are executing concurrently. That is, the execution of a transaction should not be interfered with by any other transactions executing concurrently.
- ▶ **Durability or permanency.** The changes applied to the database by a committed transaction must persist in the database. These changes must not be lost because of any failure.

Schedules of Transaction

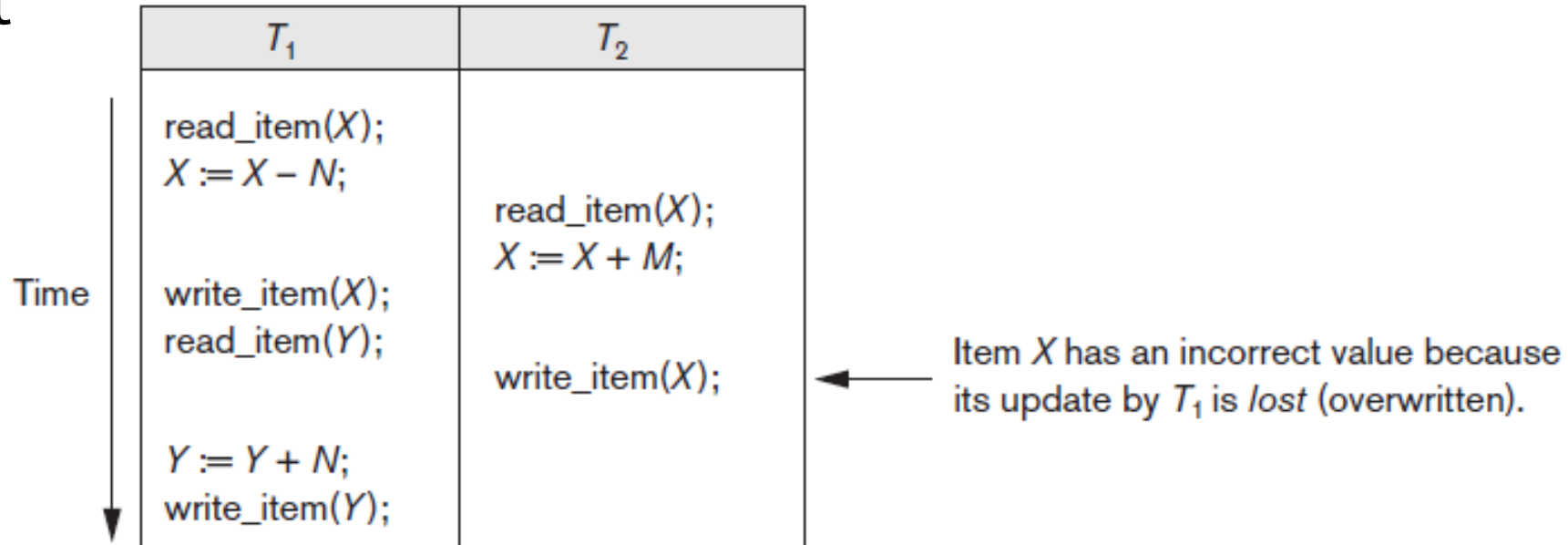
- ▶ A schedule (or history) S of n transactions $T_1, T_2, \dots, T_n$ is an ordering of the operations of the transactions.
- ▶ Operations from different transactions can be interleaved in the schedule S .
- ▶ For each transaction T_i that participates in the schedule S , the operations of T_i in S must appear in the same order in which they occur in T_i .
- ▶ For the purpose of recovery and concurrency control, mainly `read_item` and `write_item` operations of the transactions, as well as the `commit` and `abort` operations are considered here.
- ▶ A shorthand notation for describing a schedule uses the symbols `b`, `r`, `w`, `e`, `c`, and `a` for the operations `begin_transaction`, `read_item`, `write_item`, `end_transaction`, `commit`, and `abort`, respectively, and appends as a subscript the transaction id (transaction number) to each operation in the schedule

Example

- ▶ Schedule for the following transaction can be written as:

S: r1(X); r2(X); w1(X); r1(Y); w2(X); w1(Y);

Highlighted operations are conflicting operations in the schedule – read-write conflict

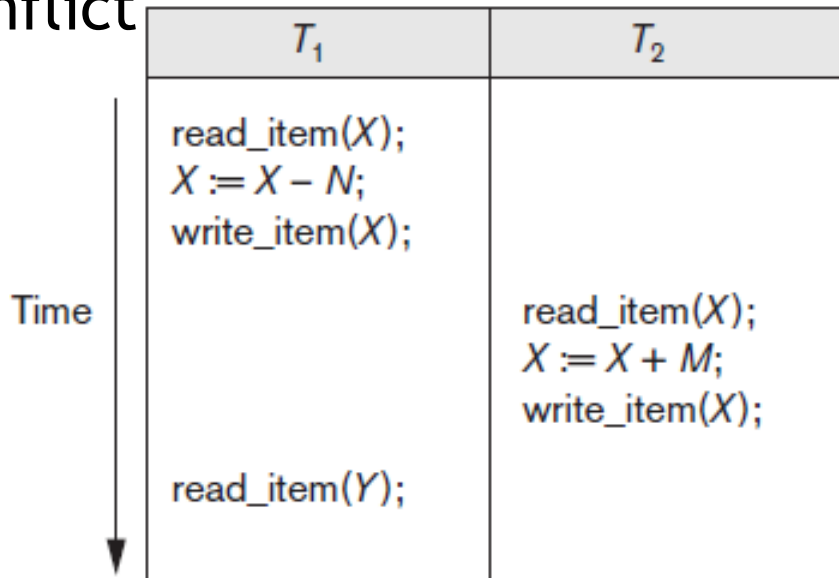


Example

- Schedule for the following transaction can be written as:

S: r1(X); w1(X); r2(X); w2(X); r1(Y); a1;

Highlighted operations are conflicting operations in the schedule – write-write conflict



Transaction T_1 fails and must change the value of X back to its old value; meanwhile T_2 has read the *temporary* incorrect value of X .

Conflicting Operations in a Schedule

- ▶ Two operations in a schedule are said to conflict if they satisfy all three of the following conditions:
 - ▶ (1) they belong to different transactions;
 - ▶ (2) they access the same item X ; and
 - ▶ (3) at least one of the operations is a `write_item(X)`.
- ▶ Two operations are conflicting if changing their order can result in a different outcome.

Complete Schedule

- ▶ A schedule S of n transactions $T_1, T_2, \dots, T_n$ is said to be a complete schedule if the following conditions hold:
 - ▶ 1. The operations in S are exactly those operations in $T_1, T_2, \dots, T_n$, including a commit or abort operation as the last operation for each transaction in the schedule.
 - ▶ 2. For any pair of operations from the same transaction T_i , their relative order of appearance in S is the same as their order of appearance in T_i .
 - ▶ 3. For any two conflicting operations, one of the two must occur before the other in the schedule

Characterizing Schedules based on Recoverability

- ▶ Once a transaction T is committed, it should never be necessary to roll back T.
- ▶ This ensures that the durability property of transactions is not violated.
- ▶ The schedules that theoretically meet this criterion are called recoverable schedules
- ▶ A schedule where a committed transaction may have to be rolled back during recovery is called nonrecoverable schedules
- ▶ and hence should not be permitted by the DBMS

The condition for a recoverable schedule is as follows

- ▶ A schedule S is recoverable if no transaction T in S commits until all transactions T' that have written some item X that T reads have committed.
- ▶ A transaction T reads from transaction T' in a schedule S if some item X is first written by T' and later read by T .
- ▶ In addition, T' should not have been aborted before T reads item X , and there should be no transactions that write X after T' writes it and before T reads it

Eg. Consider the schedule $S_{\alpha'}$ given below, which is the same as schedule S_{α} except that two commit operations have been added to S_{α} :

$S_{\alpha'} : r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); c_2; w_1(Y); c_1;$

$S_{\alpha'}$ is now called recoverable schedule, even though it suffers from the lost update problem

Example:

- ▶ Consider the two (partial) schedules S_c and S_d that follow:

▶ S_c : $r_1(X)$; $w_1(X)$; $r_2(X)$; ~~$r_1(Y)$~~ ; $w_2(X)$; c_2 ; a_1 ;  

- ▶ Non Recoverable

▶ S_d : $r_1(X)$; $w_1(X)$; $r_2(X)$; $r_1(Y)$; $w_2(X)$; $w_1(Y)$; c_1 ; c_2 ; 

- ▶ Recoverable

▶ S_e : $r_1(X)$; $w_1(X)$; $r_2(X)$; $r_1(Y)$; $w_2(X)$; $w_1(Y)$; a_1 ; a_2 ; 

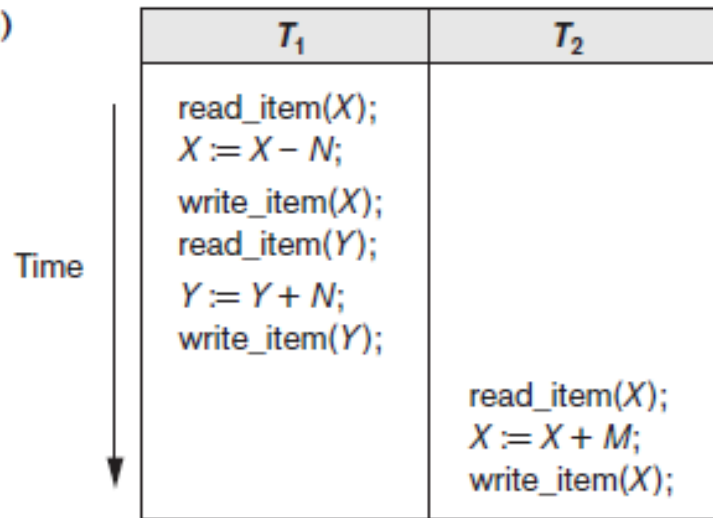
- ▶ Recoverable with cascading rollback



Characterizing Schedules Based on Serializability

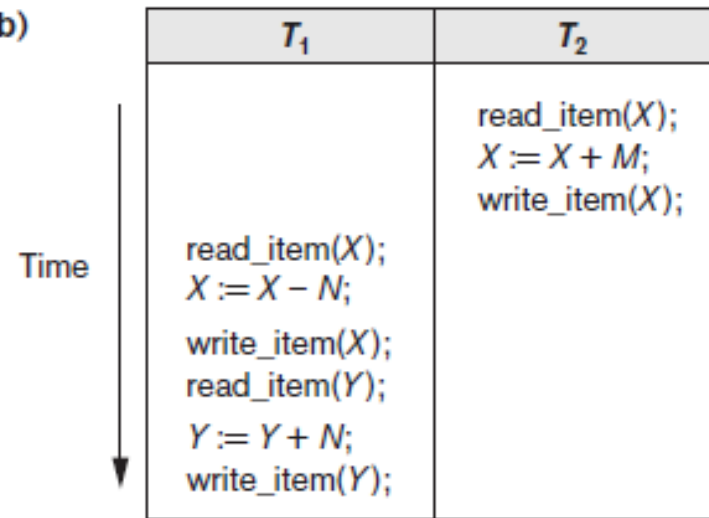
- ▶ Suppose that two users—for example, two airline reservations agents—submit to the DBMS transactions T1 and T2 at approximately the same time.
- ▶ If no interleaving of operations is permitted, there are only two possible outcomes:
 - ▶ 1. Execute all the operations of transaction T1 (in sequence) followed by all the operations of transaction T2 (in sequence).
 - ▶ 2. Execute all the operations of transaction T2 (in sequence) followed by all the operations of transaction T1 (in sequence).
- ▶ These two schedules—called serial schedules

(a)



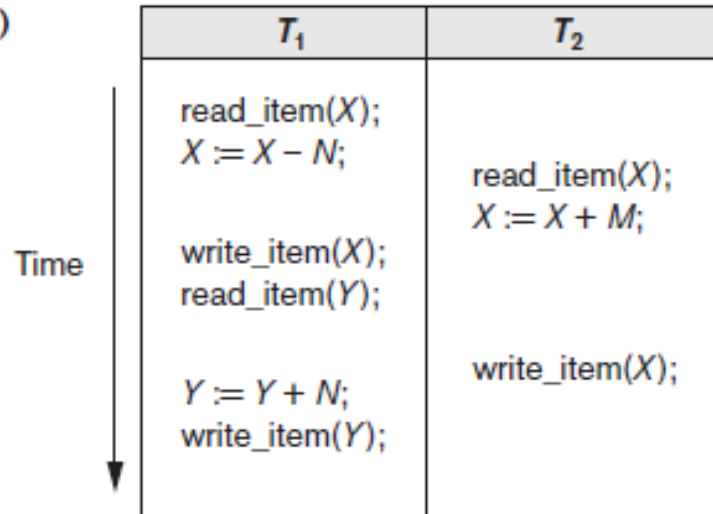
Schedule A

(b)

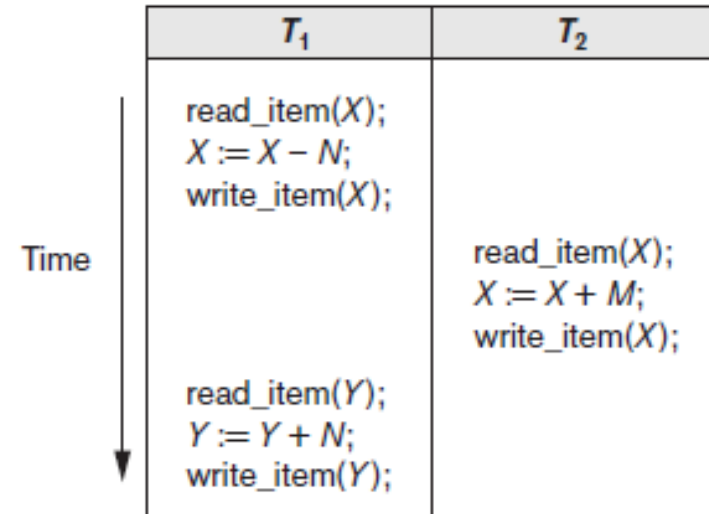


Schedule B

(c)



Schedule C



Schedule D

Schedule A and B are serial schedules

Schedule C is Non-Serializable Schedule

Schedule D is called as serializable schedule

Concurrency Control Protocols

- ▶ Concurrency control protocols ensure that concurrent execution of transactions produces the same results as some serial execution, maintaining the database's consistency (Isolation property of ACID).
- 1. Lock-Based Protocols (Two-Phase Locking - 2PL)
- 2. Timestamp Ordering Protocol
- 3. Validation-Based (Optimistic) Protocol

1. Lock-Based Protocols (Two-Phase Locking - 2PL)

This is a widely used protocol to ensure serializability. Transactions are required to request locks on data items before accessing them

- **Lock Types:**

- **Shared Lock (S-lock):** Allows a transaction to **read** a data item X (e.g., `read_item(X)`). Multiple transactions can hold S-locks on the same item simultaneously.
- **Exclusive Lock (X-lock):** Allows a transaction to **read and write** a data item X (e.g., `write_item(X)`). Only one transaction can hold an X-lock on an item at any time.

- **Two-Phase Locking (2PL):** Every transaction must obey the following rules:

- **Growing Phase:** A transaction can obtain locks, but cannot release any locks.
- **Shrinking Phase:** A transaction can release locks, but cannot obtain any new locks.

- **Strict 2PL:** A variation where a transaction holds all its exclusive (write) locks until it either **commits** or **aborts**. This ensures that the schedule is **strict** (no cascading aborts are necessary) and is often preferred in practice.

2. Timestamp Ordering Protocol

This protocol uses timestamps to manage the order of transaction operations, ensuring that conflicts are resolved according to a predefined, static ordering.

- **Timestamp ($TS(T)$):** A unique identifier assigned to a transaction T when it starts. The older transaction (smaller timestamp) has higher priority.
- **Protocol Mechanism:** For every data item X , the DBMS keeps track of:
 - **W-TS(X):** The largest timestamp of a transaction that successfully **wrote** X
 - **R-TS(X):** The largest timestamp of a transaction that successfully **read** X .
- A transaction T will only be allowed to access X if its timestamp $TS(T)$ is compatible with W-TS(X) and R-TS(X). If a request violates the ordering (e.g., an older transaction tries to write a value that a newer transaction has already read), the older transaction is typically **rolled back** and **restarted** with a new, larger timestamp.

3. Validation-Based (Optimistic) Protocol

This protocol is based on the assumption that conflicting operations are rare, thus avoiding the overhead of checking locks or timestamps with every operation. It is "optimistic" that the transaction will successfully commit.

A transaction T execution is divided into three phases:

1.Read Phase: T executes and performs all its read and write operations. All updates are applied only to local copies of data items (in the buffer).

2.Validation Phase: T is checked to see if it violates serializability. This check ensures that T and any currently committed transactions do not conflict.

3.Write Phase: If the validation is successful, the local updates are applied to the actual database; otherwise, T is **rolled back** (aborted).

Deadlock Management

Deadlock

- A **deadlock** occurs when two or more transactions wait indefinitely for each other to release locks.
- Common in **lock-based concurrency control protocols** such as **Two-Phase Locking (2PL)**.
- Results in **system halt** for those transactions unless handled by DBMS.

Deadlock Management Techniques

DBMS handles deadlocks using two approaches:

1. **Deadlock Prevention**
2. **Deadlock Detection and Recovery**

1. Deadlock Prevention

Deadlock Prevention

- DBMS ensures that **deadlocks never occur**.
- Achieved by **restricting the way transactions request locks**.
- May lead to **reduced concurrency**, but avoids deadlocks completely.

Pre-claiming of Locks

- A transaction must **acquire all required locks before execution begins**.
- If any lock is unavailable:
 - Transaction **waits** or is **aborted**.
- Prevents **hold and wait condition**.

Lock Ordering

- A **global order** is assigned to all data items.
- Transactions must request locks **only in that predefined order**.
- Prevents **circular wait condition**.

Timestamp-Based Deadlock Prevention

- Each transaction is assigned a **timestamp (TS)** when it starts.
- Older transactions have **higher priority**.
- Two popular schemes:
 - **Wait-Die**
 - **Wound-Wait**

Wait-Die Scheme

- If $TS(T_i) < TS(T_j)$ (T_i is **older**):
 - T_i is **allowed to wait** for T_j .
- If $TS(T_i) > TS(T_j)$ (T_i is **younger**):
 - T_i is **aborted (dies)** and restarted later.
- Non-preemptive scheme.

Wound-Wait Scheme

- If $TS(T_i) < TS(T_j)$ (T_i is older):
 - T_j is **aborted (wounded)** and T_i proceeds.
- If $TS(T_i) > TS(T_j)$ (T_i is younger):
 - T_i **waits**.
- Preemptive scheme.

2. Deadlock Detection and Recovery

Deadlock Detection

- DBMS allows deadlocks to occur.
- Periodically checks the system for deadlocks.
- Uses a **Wait-For Graph (WFG)**.

Wait-For Graph (WFG)

- A **directed graph** maintained by DBMS.
- Node $\rightarrow$ Transaction
- Edge $T_i \rightarrow T_j$:
 - Transaction T_i is waiting for T_j to release a lock.

Deadlock Detection Using WFG

- A cycle in the **Wait-For Graph** indicates a deadlock.
- Deadlock exists if and only if a cycle is present.

Deadlock Recovery

- When a deadlock is detected:
 - One or more transactions are **aborted (rolled back)**.
- Aborted transaction is called the **victim**.

Victim Selection Criteria

DBMS chooses a victim based on:

- Transaction that has done **least amount of work**
- Transaction **furthest from completion**
- Transaction holding **fewest locks**
- Transaction with **lowest priority**

Recovery and Logging Techniques

Recovery Management

- Recovery ensures:
 - **Atomicity** → All or nothing execution
 - **Durability** → Changes persist after commit
- Recovery is required after:
 - Transaction failure
 - System crash

The Log (Journal)

The log is a sequential file (kept on stable storage/disk) that records all database modifications. Log records are essential for undoing or redoing changes.

- Maintained on **disk**
 - Used for **UNDO and REDO operations**
 - Essential for crash recovery
-
- [start_transaction, T] - Transaction T begins
 - [write_item, T, X, old_value, new_value] - T updates data item X
 - [commit, T] - Transaction T committed successfully
 - [abort, T] - Transaction T aborted

Write-Ahead Logging (WAL) Protocol

WAL enforces **strict ordering** between log and database writes.

WAL Rule:

Log record must be written to disk **before**
The updated database block is written to disk

Importance of WAL

Ensures that **log always contains update information**

Allows DBMS to:

UNDO incomplete transactions

REDO committed transactions

Guarantees **database consistency after crash**

Checkpointing

Checkpointing is a technique used to minimize the time required for database recovery after a system crash.

- **Process:** The DBMS periodically takes a checkpoint, which involves:
 - Temporarily halting all transaction updates to the database.
 - Forcing all updated data blocks and all log records residing in main memory buffers onto the disk (flush buffers).
 - Writing a special record `[checkpoint]` to the log and forcing the log to disk.
 - Resuming normal transaction execution.
- **Benefit:** In case of a crash, the system only needs to consider the transactions active since the **last checkpoint**, significantly reducing the number of log records that must be scanned for recovery.

Recovery Using Log Records (Undo/Redo)

After a system crash, the recovery manager performs two main actions based on the log:

Operation

Purpose

Used for Transactions that...

REDO

Reapplies changes to the database.

Committed before the crash (has a [commit, T] record).

UNDO

Removes changes from the database.

Were active/uncommitted at the time of the crash.

Practical

- ▶ Oracle Transactions: A series of one or more SQL statements that are logically related or a series of operations performed on oracle table data is termed as transaction.
- ▶ Transaction is a group of events that occur between any of the following events:
 - ▶ Connection to Oracle
 - ▶ Disconnecting from oracle
 - ▶ Committing changes to the database table
 - ▶ Rollback

Using Commit

- ▶ A commit ends the current transaction and makes any changes permanent, made during transaction
- ▶ Syntax:
`COMMIT;`

Rollback

- ▶ A rollback does exactly opposite to Commit.
- ▶ It ends the transaction but undoes any changes made during the transaction
- ▶ All transaction locks acquired on tables are released
- ▶ Syntax:

```
ROLLBACK [WORK] [TO [SAVEPOINT] <SavePoint Name>];
```

Creating SavePoint

- ▶ Save point marks and saves the current point in the processing of a transaction.
- ▶ When the SavePoint is used with the rollback statement, part of a transaction can be undone
- ▶ An active SavePoint is one that is specified since the last COMMIT or ROLLBACK
- ▶ Syntax:
SAVEPOINT <SavePoint Name>